

PORTADA

Análisis y Diseño de Sistemas II

Semana 15 — Concepto de Deuda Técnica

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**Todo el código que escriben hoy
es una decisión financiera que pagarán mañana**

TRANSICIÓN

De los antipatrones a por qué existen

La semana pasada se cerró la **Unidad 5** con los antipatrones de diseño: God Object, Spaghetti Code, Golden Hammer, Copy-Paste Programming y Lava Flow. Todos comparten algo — no aparecieron porque un equipo quisiera escribir mal código. Aparecieron porque, en algún momento, alguien tomó un atajo razonable y nadie volvió a pagarlo.

Hoy se abre la **Unidad 6 — Calidad en el Diseño de Software**, con el concepto que le da nombre a ese fenómeno: la **deuda técnica**. En las próximas tres semanas se conecta con código limpio, refactorización y pruebas unitarias — las tres formas concretas de gestionarla.

CASO REAL

El origen del término: Ward Cunningham, 1992

El término "**deuda técnica**" (technical debt) lo acuñó **Ward Cunningham** — coautor del Manifiesto Ágil e inventor del primer wiki — en un reporte de experiencia de 1992 para la conferencia OOPSLA, mientras trabajaba en **WyCash**, un sistema financiero para gestión de portafolios de inversión desarrollado en Smalltalk.

Cunningham no usó la metáfora para justificar código descuidado. La usó para explicar a su equipo **por qué valía la pena reescribir** ciertas partes del sistema, aun cuando el código "funcionaba": el diseño actual, aunque válido en su momento, ya no reflejaba el entendimiento del negocio que el equipo había ganado con el tiempo.

CASO REAL

La metáfora original, en palabras de Cunningham

"Enviar código a medio terminar es como endeudarse. Un poco de deuda acelera el desarrollo, siempre que se pague pronto con una reescritura (...) El peligro ocurre cuando la deuda no se paga. Cada minuto dedicado a código que no es exactamente correcto cuenta como interés sobre esa deuda."

Es una **metáfora financiera deliberada**: así como una deuda monetaria permite comprar algo hoy a cambio de pagar intereses después, un atajo de diseño permite entregar software hoy a cambio de que el código sea más costoso de cambiar mañana. La metáfora funcionó porque le dio a los equipos de desarrollo un lenguaje que los gerentes de negocio ya entendían.

CONCEPTO

Qué es la deuda técnica

La **deuda técnica** es el costo implícito de trabajo adicional futuro que se genera al elegir una solución fácil o rápida hoy, en vez de una solución mejor que habría tomado más tiempo. No es sinónimo de "código malo": es una **relación entre una decisión de diseño y su costo diferido**.

Como toda deuda financiera tiene dos componentes:

- **El capital (principal):** el trabajo que falta por hacer para llegar al diseño correcto — la refactorización pendiente.
- **El interés:** el costo adicional que paga el equipo, sesión tras sesión, mientras esa deuda no se salda — código más difícil de entender, cambios más lentos, más bugs.

CONCEPTO

El cuadrante de Martin Fowler

Martin Fowler (el mismo autor del catálogo de code smells que se ve la próxima sesión) refinó la metáfora en 2009: no toda deuda técnica es igual. Propuso clasificarla en dos ejes independientes — si fue **deliberada** o **inadvertida**, y si se tomó de forma **prudente** o **imprudente** — formando un cuadrante de cuatro combinaciones.



CONCEPTO

Los cuatro tipos de deuda técnica

	Deliberada	Inadvertida
Imprudente	"No hay tiempo para diseñar bien" — el equipo sabe que hay un atajo y no le importa el costo futuro.	"¿Qué es diseño en capas?" — el equipo no conoce las prácticas que evitarían la deuda.
Prudente	"Hay que entregar ahora y aceptamos las consecuencias" — decisión consciente, documentada, con plan de pago.	"Ahora sabemos cómo debimos haberlo hecho" — solo se descubre en retrospectiva, con más experiencia.

Las dos casillas de la izquierda son **decisiones conscientes** del equipo; las dos de la derecha se descubren después, sin que nadie las eligiera a propósito.

CONCEPTO

Por qué la deliberada-prudente no siempre es un error

La casilla **deliberada y prudente** es la más importante para desmontar un mito: "toda deuda técnica es un error del equipo". No lo es. Un equipo puede decidir, con conocimiento completo, entregar una versión más simple de una funcionalidad para validar una hipótesis de negocio antes de invertir en la solución robusta — y eso es una **decisión de ingeniería válida**, siempre que:

- Se tome de forma **consciente y explícita**, no por omisión.
- Se **documente** qué se sacrificó y por qué (ej. un comentario `TODO` , un ticket en el backlog).
- Exista un **plan realista de pago** — no quede indefinidamente pendiente.

El problema nunca es tomar la deuda: es tomarla **sin saberlo o sin plan de pago**.

CONCEPTO

Causas comunes de deuda técnica

- **Presión de tiempo:** fechas de entrega que fuerzan a omitir diseño, pruebas o revisión de código.
- **Falta de conocimiento:** el equipo no conoce todavía SOLID, GRASP o los patrones GoF que evitarían el problema (deuda inadvertida).
- **Rotación de equipo:** quien entendía el diseño original se va, y quien llega no conoce el contexto ni las decisiones tomadas.
- **Falta de pruebas automatizadas:** sin una red de seguridad, refactorizar da miedo — y lo que da miedo, no se hace (tema central de la próxima semana).
- **Requisitos que cambian:** el diseño que era correcto para el problema original deja de encajar cuando el problema cambia.



CONCEPTO

Cómo se paga el "interés"

El interés de la deuda técnica no llega como un cargo mensual explícito — llega como **fricción**, cada vez que alguien intenta cambiar el código:

- Cada cambio nuevo toma **más tiempo** del que debería, porque hay que entender primero el desorden existente.
- Aumenta la **probabilidad de introducir bugs** al modificar código enredado o duplicado.
- El equipo empieza a **evitar tocar** ciertas zonas del sistema — exactamente el síntoma de Lava Flow visto la semana pasada.
- Con el tiempo, incluso agregar una funcionalidad pequeña requiere un esfuerzo desproporcionado.

Igual que con una deuda financiera impagada, el interés compuesto puede llegar a superar el valor del capital original: el costo de mantener el código termina siendo mayor que el que habría costado hacerlo bien desde el principio.



CONCEPTO

Cómo se mide y se hace visible

A diferencia de una deuda financiera, la deuda técnica no aparece en un estado de cuenta — hay que hacerla visible deliberadamente. Algunas formas comunes:

- **Herramientas de análisis estático** como **SonarQube**, que escanean el código y reportan métricas de complejidad, duplicación y cobertura de pruebas, además de estimar el "tiempo de remediación" en horas.
- **Code smells** (síntomas que se ven en detalle la próxima sesión) como señal indirecta: mientras más smells acumula una clase, más deuda probablemente arrastra.
- **Revisión de código (code review)**, donde el equipo señala atajos antes de que se fusionen al proyecto.
- **Backlog explícito de deuda técnica**: registrar cada atajo tomado como un ítem visible, igual que una historia de usuario.



TENSIÓN CONCEPTUAL

"Deuda cero" no es la meta

Un error común es pensar que el objetivo de un buen equipo es **nunca** tener deuda técnica. En la práctica, eso ni es posible ni es deseable: incluso el mejor entendimiento del negocio de hoy va a quedar parcialmente obsoleto mañana, y siempre habrá presión legítima de tiempo de mercado.

La meta realista no es cero deuda, es **deuda gestionada**: tomarla de forma consciente cuando aporta valor, hacerla visible, y pagarla antes de que su interés supere lo que aportó. Un equipo maduro no es el que nunca toma atajos — es el que sabe exactamente cuáles tomó y cuándo piensa pagarlos.

INTERACCIÓN

Clasifiquen estos tres casos en el cuadrante de Fowler

En el chat, para cada caso indiquen si la deuda técnica es deliberada o inadvertida, y prudente o imprudente:

1. Un equipo entrega una funcionalidad copiando y pegando código de otra pantalla similar, porque nunca se les ocurrió que se pudiera reutilizar.
2. Un equipo decide, en una reunión, entregar sin pruebas automatizadas dos historias de esta iteración para cumplir la fecha del demo, y lo anota en el backlog para corregirlo la próxima semana.
3. Un equipo sabe que su diseño actual no escala, pero decide seguir agregando funcionalidades encima sin decírselo a nadie, "porque total ya funciona".

5 minutos · respondan en el chat

CONCEPTO · RELACIÓN CON EL PROYECTO

Deuda técnica y el proyecto de Fábrica Escuela

A partir de esta semana, cada equipo debe empezar a **auditar su propia deuda técnica** en el proyecto que vienen construyendo desde la semana 2. No es un ejercicio teórico aislado: es revisar con honestidad las decisiones de diseño ya tomadas —muchas de ellas, razonables en su momento— y hacer visible lo que quedó pendiente.

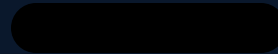
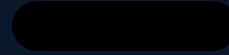
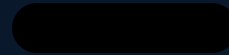
- ¿Qué atajos tomaron a propósito, y quedaron documentados?
- ¿Qué partes del diseño ya no reflejan lo que el equipo entiende hoy del problema?
- ¿Qué antipatrones de la semana pasada (God Object, Spaghetti Code) son, en el fondo, deuda técnica acumulada?



SÍNTESIS

Ideas clave de la sesión

1. La deuda técnica es una **metáfora financiera** acuñada por Ward Cunningham en 1992: capital (trabajo pendiente) más interés (costo de no pagarlo).
2. El cuadrante de Fowler la clasifica en **deliberada/inadvertida** y **prudente/imprudente** — no toda deuda es un error.
3. Las causas más comunes son presión de tiempo, falta de conocimiento, rotación de equipo y falta de pruebas.
4. El interés se paga como **fricción creciente**: cambios más lentos, más riesgo de bugs, zonas del código que nadie quiere tocar.
5. Se hace visible con herramientas como SonarQube, code smells y un backlog explícito de deuda.



CIERRE

Antes de la próxima sesión

Empiecen a identificar, de manera informal, uno o dos lugares de su proyecto donde reconocen deuda técnica — no hace falta documentarlo todavía formalmente, solo tenerlo presente. La próxima sesión se aborda la primera herramienta concreta para pagarla: **código limpio**, empezando por nombres, funciones y comentarios, según "Clean Code" de Robert C. Martin.

